`SaveChangesAsync` is called, consists of an SQL `DELETE` statement that deletes it from the database.

There is actually no need to retrieve the entity at all. You can use what is known as a stub to represent the entity to be removed. A *stub* is an entity that only has its key value assigned. Assume you have already modified this page to work with a constrained route parameter representing the key value, instead of a nullable handler parameter. The next listing shows how to use a stub to represent the entity to be deleted in the `OnPostAsync` method.

Listing 8.34 Deleting an entity represented by a stub

```
public async Task<IActionResult> OnPostAsync()
{
    var property = new Property { Id = Id };      ❶
    _context.Remove(property );                   ❷
    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}
```

❶ Create a stub, only assigning its key value.

❷ Pass the stub to the DbContext.Remove method, which marks the entity as Deleted.

Taking this approach, you significantly reduce the complexity of the code and halve the number of database calls required to achieve your objective. Note that you have also used the `DbContext.Remove` method instead of the `DbSet.Remove` method the scaffolder generates. Like the `DbContext` `Add` and `Update` methods, the `Remove` method is new to EF Core and helps you reduce code.

When you delete an entity, all dependent data is either lost or orphaned. Depending on how your foreign keys are set up, the delete operation cascades to all dependent data, and it is either also removed, or its foreign key values are updated to `null`, resulting in orphaned data. If the